INSIDE STLC ACADEMY

PRACTICAL TEST AUTOMATION WITH PLAYWRIGHT

Visual & Screenshot Testing Guide

Catch unintended visual changes with snapshot testing.

Introduction

Visual regression testing catches changes functional tests miss — layout breaks, missing styles, shifted components. Playwright has built-in snapshot comparison. This guide shows you how to use it well.

How It Works

- await expect(page).toHaveScreenshot() captures and compares an image.
- The first run creates a baseline; later runs compare against it.
- Update baselines deliberately with --update-snapshots.

Worked Example

- await expect(page).toHaveScreenshot('dashboard.png');
- await expect(page.getByRole('navigation')).toHaveScreenshot('nav.png');

Keeping Visual Tests Stable

Problem	Mitigation
Animations	Disable or wait for them to finish
Dynamic data/dates	Mask or stub them
Font rendering differences	Run in CI/containers consistently
Full-page noise	Snapshot a component, not the whole page

Practical Exercise

Add a screenshot assertion for one stable component. Make a small CSS change and confirm the test fails, then update the baseline intentionally.

Best Practices

- Snapshot stable components, not noisy full pages.
- Control dynamic content before snapshotting.
- Update baselines deliberately and review the diff.

Common Mistakes

- Snapshotting pages full of changing data.
- Blindly updating baselines to make tests pass.
- Different rendering locally vs CI causing false diffs.

> **INSIDE STLC PRO TIP**
>
> Run and store visual baselines in the same environment (usually CI/containers) every time. Local font and rendering differences are the number-one cause of false visual failures.